详细设计说明书 (Detailed Design Specification)

地牢探险游戏 (Dungeon Adventure Game)

文档信息

项目	内容
项目名称	Dungeon Adventure Game（地牢探险游戏）
文档版本	v2.0 (完整版)
编写日期	2026年4月8日
编写人	开发团队
审核人	Scrum Master
状态	☑ 已审核通过

目录

1. 引言
2. 系统架构概述
3. 类详细设计
    - 3.1 Entity 基类
    - 3.2 Player 类
    - 3.3 Enemy 类
    - 3.4 Map 类
    - 3.5 Battle 类
    - 3.6 Renderer 类
    - 3.7 Game 类
4. 核心算法 N-S 盒图
5. 跨类调用关系
6. 数据流说明
7. 附录

---

1. 引言

1.1 编写目的

本文档为地牢探险游戏 Sprint 2 重构版的详细设计说明书，旨在：

- 归档重构后核心控制类的内部关键算法
- 规范化跨类调用的方法签名细节
- 为后续开发、测试、维护提供技术参考

1.2 适用范围

本文档适用于：

- 开发人员：理解系统内部设计和算法实现
- 测试人员：编写测试用例和测试数据
- 维护人员：定位问题和扩展功能
- 项目管理人员：评估代码质量和进度

1.3 术语定义

术语	定义
N-S 盒图	Nassi-Shneiderman Diagram，Nassi-Shneiderman 图（盒图）
McCabe 圈复杂度	衡量程序复杂度的指标，计算公式：判断总数 + 1
上帝类	承担过多职责的单一类，违反单一职责原则
封装	隐藏内部实现，通过接口暴露功能
继承	子类复用父类属性和方法
多态	同一接口不同实现

---

2. 系统架构概述

2.1 架构设计原则

本次重构基于面向对象设计三大原则：

原则	实现方式	解决的问题
封装	拆分上帝类为 5 个独立对象，所有属性 protected/private	深层耦合、破坏封装
继承	设计 Entity 父类，Player/Enemy 继承	代码重复
多态	纯虚函数 getSymbol()/getName()，渲染系统自动适配	设计僵化、硬编码

2.2 类关系图

```
┌──────────────┐
│    Game      │  ← 游戏流程控制（协调器）
│ （协调器）    │
└──────┬───────
```

```
        | 组合
        ├──────────┬──────────┬──────────┬──────────┐
        |          |          |          |          |
┌───────▼───────┐ ┌─▼─────┐ ┌─▼─────┐ ┌─▼─────┐ ┌─▼─────┐
|     Map       | |Player | |Enemy  | |Battle | |Renderer|
|  (地图数据)   | |(实体) | |(实体) | |(战斗) | |(渲染) |
└───────────────┘ └───┬───┘ └───┬───┘ └───────┘ └───────┘
                      |         |
                      └────┬────┘
                           |
                      ┌────▼────┐
                      |  Entity |  ← 基类 (继承)
                      | (实体)  |
                      └─────────┘
```

2.3 模块职责划分

模块	职责	对应 DFD 处理
Game	游戏主循环、菜单系统、输入处理、状态管理	游戏逻辑控制 (4)
Player	玩家移动、碰撞检测、足迹管理	玩家输入处理 (3)
Enemy	敌人 AI（追击/巡逻）、足迹管理	游戏逻辑控制 (4)
Map	地图生成、碰撞检测、迷雾系统	游戏初始化 (1)
Battle	回合制战斗、伤害计算、战斗日志	回合战斗 (4.4)
Renderer	控制台渲染、颜色管理、UI 元素	界面渲染 (2)

---

3. 类详细设计

3.1 Entity 基类

3.1.1 类概述

项目	说明
类名	`Entity`
文件	`entity.h`, `entity.cpp`
职责	封装玩家和敌人的公共属性和方法
访问修饰符	`public` (基类)
继承关系	被 `Player` 和 `Enemy` 继承

3.1.2 属性定义

属性名	类型	访问控制	说明
`x`	`int`	protected	X 坐标
`y`	`int`	protected	Y 坐标
`hp`	`int`	protected	当前生命值
`maxHp`	`int`	protected	最大生命值
`attack`	`int`	protected	攻击力

3.1.3 方法签名

方法名	返回类型	参数	访问控制	说明
`Entity()`	构造函数	`int x, int y, int hp, int attack`	public	初始化实体
`~Entity()`	析构函数	无	public virtual	虚析构函数
`getX()`	`int`	无	public const	获取 X 坐标
`getY()`	`int`	无	public const	获取 Y 坐标
`setPosition()`	`void`	`int x, int y`	public	设置位置
`getHp()`	`int`	无	public const	获取当前 HP
`getMaxHp()`	`int`	无	public const	获取最大 HP
`setHp()`	`void`	`int hp`	public	设置 HP（自动边界检查）
`getAttack()`	`int`	无	public const	获取攻击力
`setAttack()`	`void`	`int attack`	public	设置攻击力
`takeDamage()`	`void`	`int damage`	public virtual	受到伤害
`isAlive()`	`bool`	无	public const	是否存活
`getHealthBar()`	`string`	无	public virtual	获取血量条字符串
`getSymbol()`	`char`	无	public pure virtual	获取实体符号（纯虚）
`getName()`	`string`	无	public pure virtual	获取实体名称（纯虚）

3.1.4 前置/后置条件

方法	前置条件	后置条件
`setHp(hp)`	`hp >= 0`	`0 <= this.hp <= maxHp`
`takeDamage(damage)`	`damage >= 0`	`this.hp >= 0`
`isAlive()`	无	返回 `hp > 0`

3.2 Player 类

3.2.1 类概述

项目	说明
类名	`Player`
文件	`player.h`, `player.cpp`
职责	玩家移动逻辑、状态管理、碰撞检测、足迹管理
继承关系	继承 `Entity`

3.2.2 属性定义

属性名	类型	访问控制	说明
`footprints`	`vector<pair<int,int>>`	private	玩家足迹列表
`MAX_FOOTPRINTS`	`size_t`	private static const	最大足迹数（5）

3.2.3 方法签名

方法名	返回类型	参数	访问控制	说明
`Player()`	构造函数	无	public	初始化玩家（默认位置中心）
`move()`	`bool`	`int dx, int dy, const Map& map, const vector<Enemy>& enemies`	public	移动玩家
`reset()`	`void`	无	public	重置玩家状态
`getStatus()`	`string`	无	public const	获取状态字符串
`getSymbol()`	`char`	无	public override	返回 '@'
`getName()`	`string`	无	public override	返回 "Player"
`addFootprint()`	`void`	`int x, int y`	public	添加足迹
`getFootprints()`	`const vector<pair<int,int>>&`	无	public const	获取足迹列表
`clearFootprints()`	`void`	无	public	清空足迹

3.2.4 前置/后置条件

方法	前置条件	后置条件
`move(dx, dy, map, enemies)`	`map` 已初始化，`enemies` 非空	返回 true 时位置已更新，足迹已添加
`reset()`	无	位置=中心，HP=20，ATK=5，足迹清空

3.3 Enemy 类

3.3.1 类概述

项目	说明
类名	`Enemy`
文件	`enemy.h`，`enemy.cpp`
职责	敌人 AI（追击/巡逻）、足迹管理、敌人生成
继承关系	继承 `Entity`

3.3.2 属性定义

属性名	类型	访问控制	说明
`id`	`int`	private	敌人唯一标识
`footprints`	`vector<pair<int,int>>`	private	敌人足迹列表
`MAX_FOOTPRINTS`	`size_t`	private static const	最大足迹数（5）

3.3.3 方法签名

方法名	返回类型	参数	访问控制	说明
`Enemy()`	构造函数	无	public	默认敌人（HP=10, ATK=3）
`Enemy()`	构造函数	`int x, int y, int hp, int atk, int id`	public	自定义敌人
`moveTowardsPlayer()`	`bool`	`int px, int py, const Map& map, const vector<Enemy>& enemies`	public	追击玩家
`randomMove()`	`bool`	`const Map& map, const vector<Enemy>& enemies`	public	随机移动
`getSymbol()`	`char`	无	public override	返回 'E'
`getName()`	`string`	无	public override	返回 "Enemy X"
`getId()`	`int`	无	public const	获取 ID
`addFootprint()`	`void`	`int x, int y`	public	添加足迹
`getFootprints()`	`const vector<pair<int,int>>&`	无	public const	获取足迹列表
`clearFootprints()`	`void`	无	public	清空足迹
`isEnemyAt()`	`bool`	`int x, int y, const vector<Enemy>& e, int excludeId`	private static	检查位置是否有敌人

3.3.4 前置/后置条件

方法	前置条件	后置条件
`moveTowardsPlayer(...)`	玩家在地图内	返回 true 时更接近玩家，足迹已添加
`randomMove(...)`	地图可通行	返回 true 时位置已更新，足迹已添加

3.4 Map 类

3.4.1 类概述

项目	说明
类名	`Map`
文件	`map.h`, `map.cpp`
职责	地图生成、碰撞检测、迷雾系统、探索管理

3.4.2 属性定义

属性名	类型	访问控制	说明
`WIDTH`	`int`	public static const	地图宽度（20）
`HEIGHT`	`int`	public static const	地图高度（20）
`VIEW_RADIUS`	`int`	public static const	视野半径（6）
`grid`	`char[20][20]`	private	地图网格
`explored`	`bool[20][20]`	private mutable	探索标记

3.4.3 方法签名

方法名	返回类型	参数	访问控制	说明
`Map()`	构造函数	无	public	生成地图
`isWalkable()`	`bool`	`int x, int y`	public const	检查是否可通行
`getTile()`	`char`	`int x, int y`	public const	获取地形字符
`isVisible()`	`bool`	`int px, int py, int x, int y`	public const	检查是否可见
`isExplored()`	`bool`	`int x, int y`	public const	检查是否已探索
`updateExplored()`	`void`	`int px, int py`	public const	更新探索区域

3.4.4 前置/后置条件

方法	前置条件	后置条件
`isWalkable(x, y)`	无	返回 `x,y` 在边界内且为 '.'
`isVisible(px, py, x, y)`	坐标在地图内	返回欧几里得距离 ≤ VIEW_RADIUS
`updateExplored(px, py)`	玩家在地图内	圆形区域内 explored 标记为 true

3.5 Battle 类

3.5.1 类概述

项目	说明
类名	`Battle`
文件	`battle.h`, `battle.cpp`
职责	回合制战斗计算、伤害计算、战斗日志管理
依赖类	`Player`, `Enemy`, `Colors`
被调用类	`Game`

3.5.2 属性定义

属性名	类型	访问控制	说明
`MAX_COMBAT_LOG_SIZE`	`size_t`	private static const	最大战斗日志数（10）

3.5.3 方法签名

方法名	返回类型	参数	访问控制	说明
`Battle()`	构造函数	无	public	初始化战斗系统
`executeCombat()`	`bool`	`Player& p, Enemy& e, bool first, vector<string>& log`	public	执行回合制战斗
`calculateDamage()`	`int`	`int atk, int def=0`	public const	计算伤害
`addCombatLog()`	`void`	`const string& msg, vector<string>& log`	public const	添加战斗日志

3.5.4 前置/后置条件

方法	前置条件	后置条件
`executeCombat(...)`	`p.isAlive()==true`, `e.isAlive()==true`	返回 true 时 `e.isAlive()==false`，返回 false 时 `p.isAlive()==false`
`calculateDamage(atk, def)`	`atk>=0, def>=0`	返回值 `>= 1`
`addCombatLog(msg, log)`	`log` 已初始化	`log.size() <= MAX_COMBAT_LOG_SIZE`

3.6 Renderer 类

3.6.1 类概述

项目	说明
类名	`Renderer`
文件	`renderer.h`, `renderer.cpp`
职责	控制台渲染、颜色管理、UI 元素绘制
依赖类	`Map`, `Player`, `Enemy`, `Colors`
被调用类	`Game`

3.6.2 属性定义

属性名	类型	访问控制	说明
无	-	-	无状态类（纯工具类）

3.6.3 方法签名

方法名	返回类型	参数	访问控制	说明
`Renderer()`	构造函数	无	public	初始化渲染器
`renderGame()`	`void`	`const Map& m, const Player& p, const vector<Enemy>& e`	public const	渲染完整游戏画面
`renderMap()`	`void`	`const Map& m, const Player& p, const vector<Enemy>& e`	public const	渲染地图（含迷雾、足迹）
`renderPlayerStatus()`	`void`	`const Player& p, int alive, int total, const vector<string>& log, const vector<Enemy>* e=nullptr`	public const	渲染玩家状态
`renderCombatHeader()`	`void`	`const Enemy& e, const Player& p, bool first`	public const	渲染战斗画面
`drawLine()`	`void`	`int width=50`	public const	绘制分隔线
`drawCenteredText()`	`void`	`const string& text, int width=50`	public const	绘制居中文字
`clearScreen()`	`void`	无	public const	清屏

3.6.4 前置/后置条件

方法	前置条件	后置条件
`renderGame(...)`	地图、玩家、敌人已初始化	控制台已渲染完整画面
`renderPlayerStatus(...)`	`alive >= 0`, `total >= alive`	控制台已输出玩家状态

3.7 Game 类

3.7.1 类概述

项目	说明
类名	`Game`
文件	`game.h`, `game.cpp`
职责	游戏主循环、菜单系统、输入处理、状态管理、模块协调
依赖类	`Map`, `Player`, `Enemy`, `Battle`, `Renderer`, `Keyboard`, `Colors`
被调用类	`main()`

3.7.2 属性定义

属性名	类型	访问控制	说明
`map`	`Map`	private	地图对象
`player`	`Player`	private	玩家对象
`enemies`	`vector<Enemy>`	private	敌人列表
`battle`	`Battle`	private	战斗对象（委托）
`renderer`	`Renderer`	private	渲染器（委托）
`isRunning`	`bool`	private	游戏运行标志
`result`	`GameResult`	private	游戏结果
`difficulty`	`Difficulty`	private	难度
`settings`	`DifficultySettings`	private	难度设置
`combatLog`	`vector<string>`	private	战斗日志
`CHASE_RADIUS`	`int`	private static const	敌人追击半径（5）

3.7.3 方法签名

方法名	返回类型	参数	访问控制	说明
`Game()`	构造函数	无	public	初始化游戏
`run()`	`bool`	无	public	运行游戏主循环
`initGame()`	`void`	无	private	初始化游戏状态
`render()`	`void`	无	private	渲染画面（委托给 Renderer）
`handleInput()`	`bool`	无	private	处理玩家输入
`updateEnemies()`	`void`	无	private	更新敌人 AI
`checkGameOver()`	`void`	无	private	检查游戏结束
`showMainMenu()`	`bool`	无	private	显示主菜单
`showEndScreen()`	`bool`	无	private	显示结算画面
`confirmQuit()`	`bool`	无	private	确认退出
`countAliveEnemies()`	`int`	无	private const	统计存活敌人
`showPlayerStatus()`	`void`	无	private	显示玩家状态
`combat()`	`void`	`size_t idx, bool first=false`	private	触发战斗

3.7.4 前置/后置条件

方法	前置条件	后置条件
`run()`	无	返回 true 时返回主菜单，false 时退出游戏
`handleInput()`	游戏运行中	返回 false 时玩家选择退出
`combat(idx, first)`	`idx < enemies.size()`, `enemies[idx].isAlive()`	战斗结束，日志已更新

---

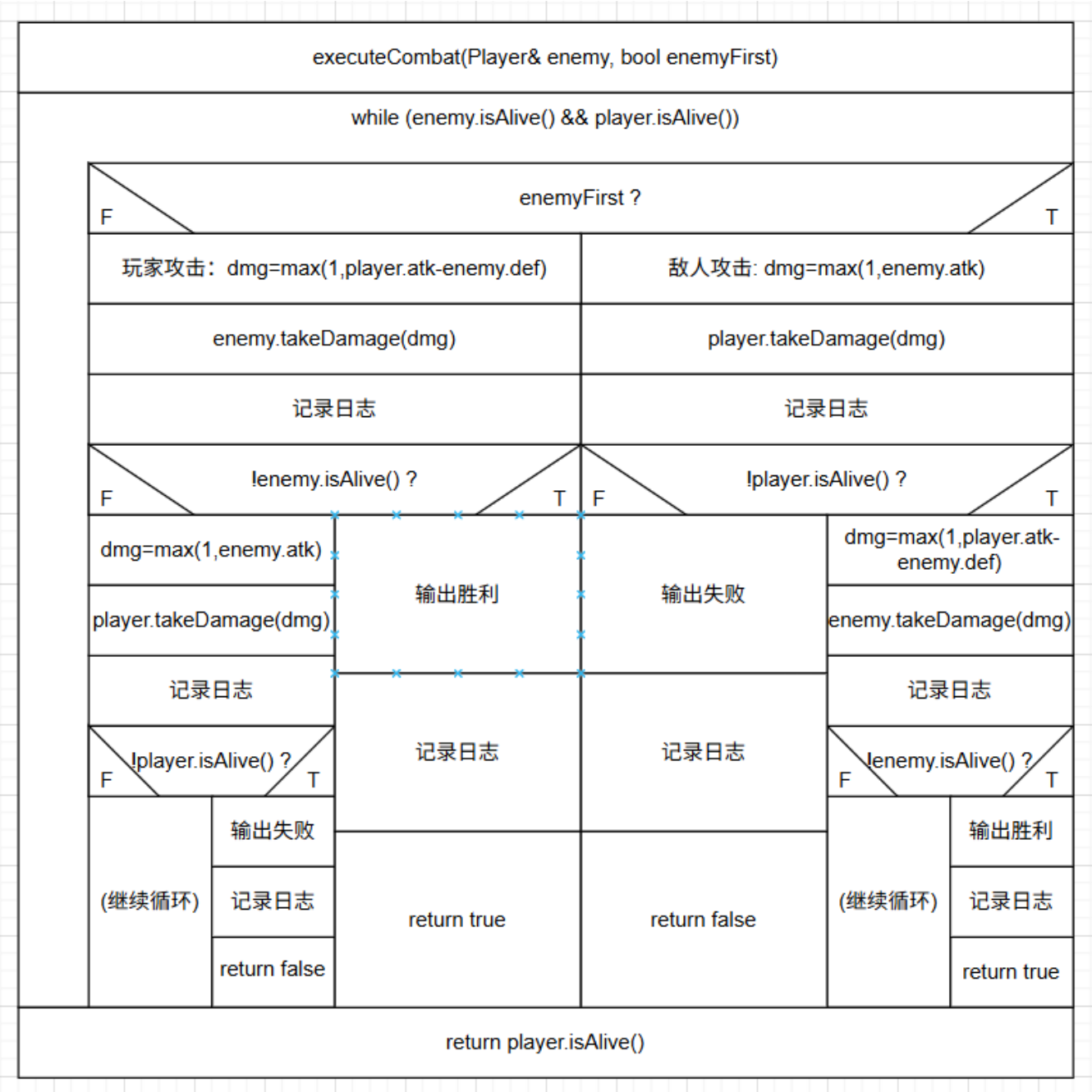
4. 核心算法 N-S 盒图

4.1 算法清单

序号	算法名称	对应方法	所属类	圈复杂度
1	回合制战斗执行算法	`executeCombat()`	Battle	11
2	主菜单选择算法	`showMainMenu()`	Game	11
3	游戏主循环算法	`run()`	Game	5
4	玩家输入处理算法	`handleInput()`	Game	8
5	敌人 AI 更新算法	`updateEnemies()`	Game	7

4.2 算法 1：回合制战斗执行算法

对应方法：`Battle::executeCombat(Player& player, Enemy& enemy, bool enemyFirst, std::vector<std::string>& combatLog)`

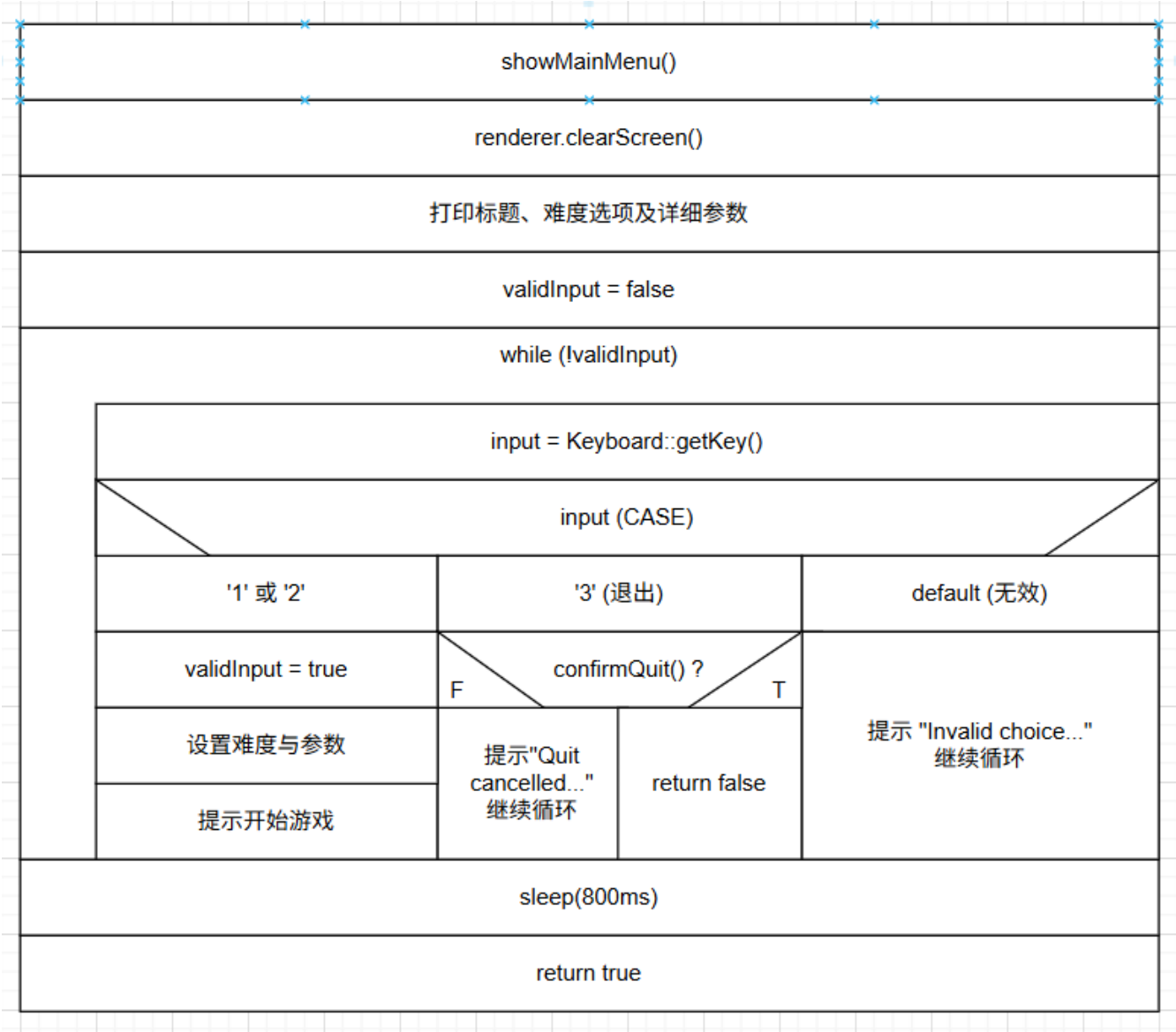


局部变量说明:

变量名	类型	作用域	说明
`enemyDamage`	`int`	分支局部	敌人对玩家造成的伤害
`playerDamage`	`int`	分支局部	玩家对敌人造成的伤害

4.3 算法 2：主菜单选择算法

对应方法: `Game::showMainMenu()`

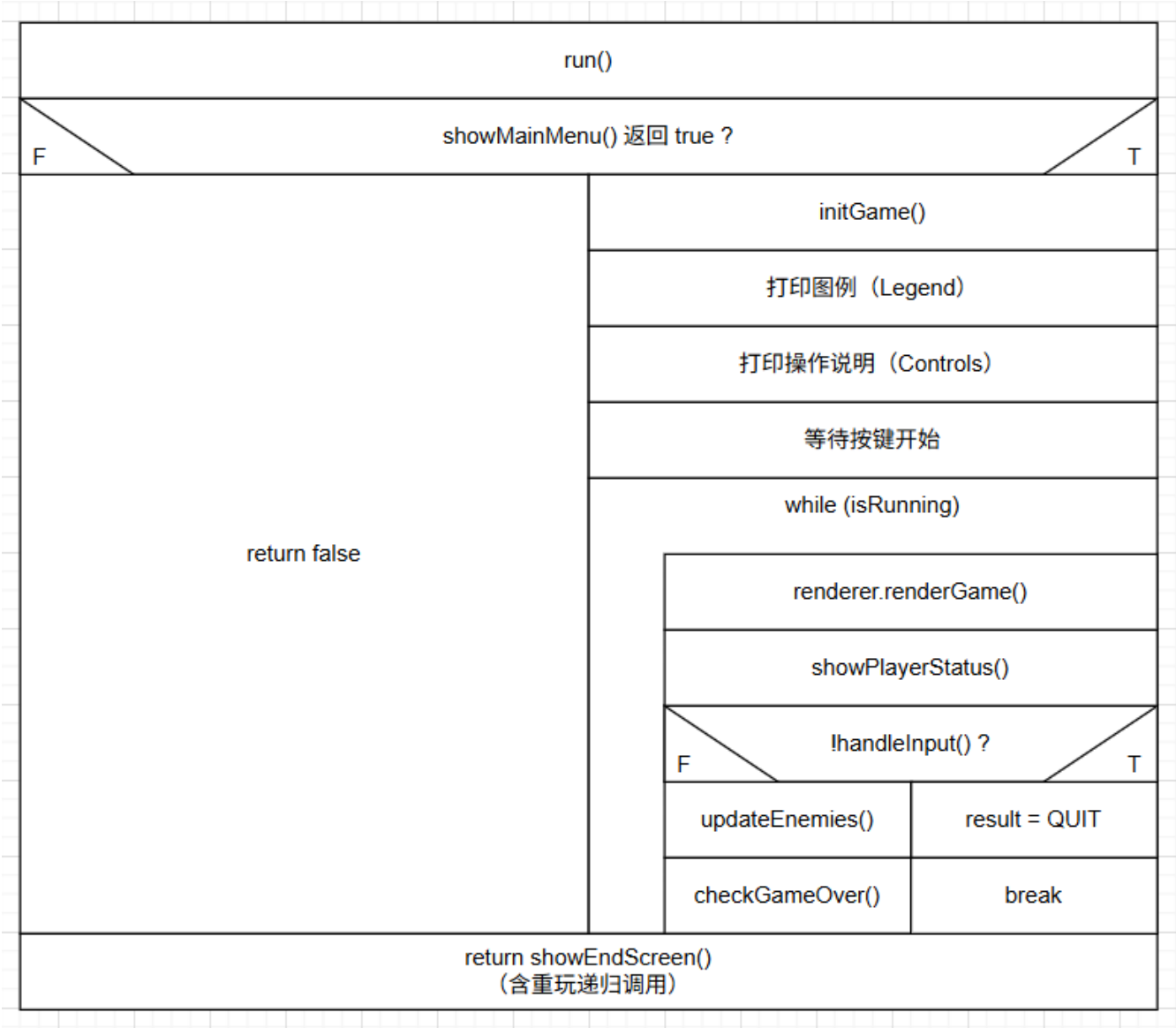


局部变量说明：

变量名	类型	作用域	初始值	说明
`validInput`	`bool`	方法局部	`false`	输入有效性标志
`input`	`char`	方法局部	无	用户输入字符

4.4 算法 3：游戏主循环算法

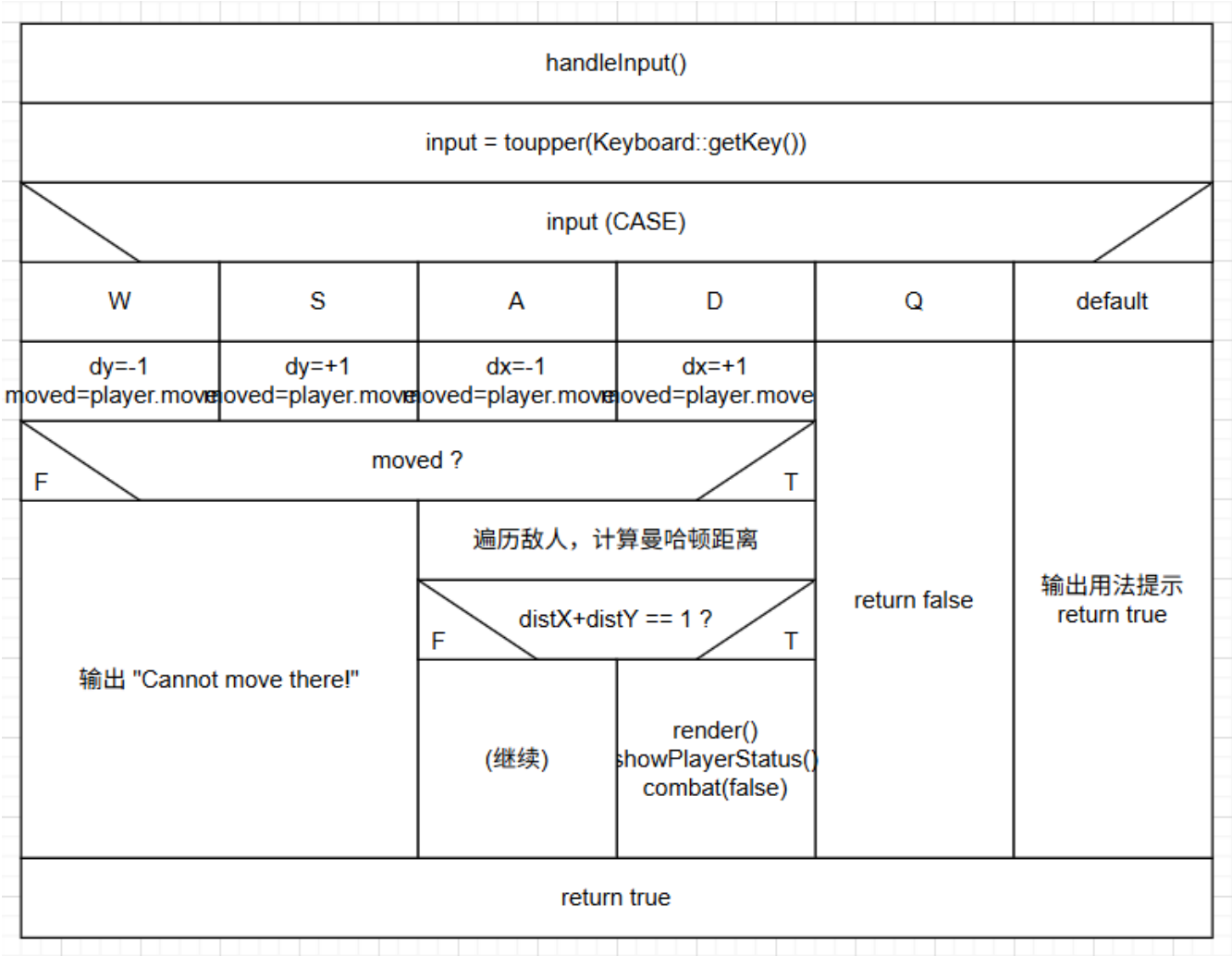
对应方法：`Game::run()`



局部变量说明：无（状态由类属性维护）

4.5 算法 4：玩家输入处理算法

对应方法：`Game::handleInput()`

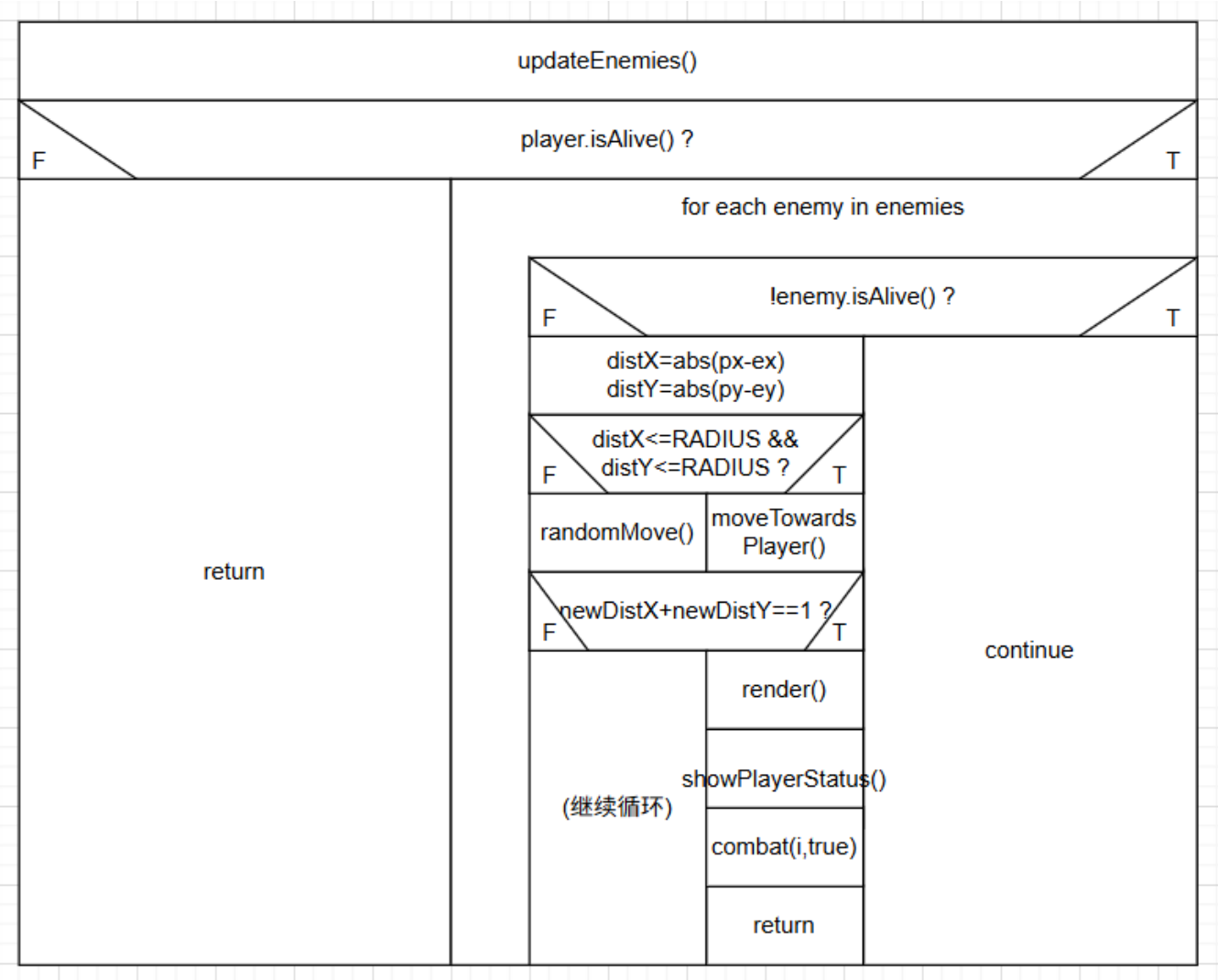


局部变量说明：

变量名	类型	作用域	说明
`input`	`char`	方法局部	用户输入字符
`dx, dy`	`int`	方法局部	移动方向
`moved`	`bool`	方法局部	移动是否成功
`adjacentToEnemy`	`bool`	分支局部	是否邻接敌人
`enemyIndex`	`size_t`	分支局部	邻接敌人索引

4.6 算法 5：敌人 AI 更新算法

对应方法：`Game::updateEnemies()`



局部变量说明:

变量名	类型	作用域	说明
`distX, distY`	`int`	循环局部	敌人与玩家 XY 距离
`newDistX, newDistY`	`int`	循环局部	移动后 XY 距离

---

5. 跨类调用关系

5.1 调用关系总览

```
┌──────────────┐
│    main()    │
└──────┬───────┘
       │ 调用
       ▼
┌──────────────┐          ┌──────────────┐          ┌──────────────┐
│    Game      │──────────▶│  Renderer    │──────────▶│   Colors     │
│ （主控制器）  │  调用    │ （渲染模块）  │  调用    │ （颜色输出）  │
```

```
└──────────┬──────────┘          └──────────┘          └──────────┘
           │
           ├──────────────▶┌──────────┐          ┌──────────┐
           │   调用        │  Battle  │──────────▶│  Player  │
           │               │ (战斗模块) │  调用    │ (玩家对象) │
           │               └──────────┘          └──────────┘
           │                     │
           │                     ▼
           │               ┌──────────┐
           ├──────────────▶│  Enemy   │
           │   调用        │ (敌人对象) │
           │               └──────────┘
           │
           ├──────────────▶┌──────────┐
           │   调用        │   Map    │
           │               │ (地图模块) │
           │               └──────────┘
           │
           └──────────────▶┌──────────┐
               调用        │ Keyboard │
                           │ (键盘输入) │
                           └──────────┘
```

5.2 详细调用链

调用链 1：游戏启动流程

```
main()
  → Game::run()
      → Game::showMainMenu()
          → Renderer::clearScreen()
          → Renderer::drawLine()
          → Renderer::drawCenteredText()
          → Keyboard::getKey()
          → Game::confirmQuit()
      → Game::initGame()
          → Player::reset()
          → spawnEnemies()
      → Renderer::renderGame()
          → Renderer::renderMap()
              → Map::updateExplored()
              → Map::isVisible()
              → Map::isExplored()
              → Map::getTile()
              → Player::getX()
              → Player::getY()
              → Enemy::getX()
              → Enemy::getY()
              → Enemy::isAlive()
              → Enemy::getSymbol()
              → Enemy::getId()
              → Enemy::getFootprints()
```

```
            → Player::getFootprints()
    → Game::showPlayerStatus()
        → Renderer::renderPlayerStatus()
            → Player::getStatus()
            → Player::getHp()
            → Enemy::isAlive()
            → Enemy::getSymbol()
            → Enemy::getName()
            → Enemy::getId()
```

调用链 2：战斗流程

```
Game::handleInput() / Game::updateEnemies()
  → Game::combat(enemyIndex, enemyFirst)
      → Renderer::renderCombatHeader()
          → Renderer::drawLine()
          → Renderer::drawCenteredText()
          → Enemy::getName()
          → Enemy::getHealthBar()
          → Enemy::getAttack()
          → Player::getHealthBar()
      → Battle::executeCombat()
          → Battle::calculateDamage()
          → Player::takeDamage()
          → Enemy::takeDamage()
          → Player::isAlive()
          → Enemy::isAlive()
          → Player::getAttack()
          → Enemy::getAttack()
          → Player::getHealthBar()
          → Enemy::getHealthBar()
          → Battle::addCombatLog()
      → Keyboard::getKey()
```

调用链 3：敌人 AI 流程

```
Game::updateEnemies()
  → Player::isAlive()
  → Enemy::isAlive()
  → Enemy::moveTowardsPlayer()
      → Map::isWalkable()
      → Enemy::isEnemyAt()
          → Enemy::isAlive()
          → Enemy::getId()
          → Enemy::getX()
          → Enemy::getY()
      → Enemy::addFootprint()
  → Enemy::randomMove()
      → Map::isWalkable()
      → Enemy::isEnemyAt()
      → Enemy::addFootprint()
  → Game::combat()
```

6. 数据流说明

6.1 战斗流程数据流

```
玩家输入 ──▶ Game::handleInput()
                 |
                 ▼ 检测到邻接敌人
           Game::combat(enemyIndex, enemyFirst)
                 |
                 ├─▶ Renderer::renderCombatHeader()  // 渲染战斗画面
                 |       |
                 |       └─▶ 输出敌人信息、血量、先手标志
                 |
                 └─▶ Battle::executeCombat(player, enemy, enemyFirst, combatLog)
                         |
                         ├─▶ Battle::calculateDamage()  // 计算伤害
                         |       |
                         |       └─▶ return damage (int)
                         |
                         ├─▶ Player::takeDamage(damage)  // 玩家扣血
                         |       |
                         |       └─▶ player.hp -= damage
                         |
                         ├─▶ Enemy::takeDamage(damage)  // 敌人扣血
                         |       |
                         |       └─▶ enemy.hp -= damage
                         |
                         └─▶ Battle::addCombatLog(message, combatLog)  // 记录日志
                                 |
                                 └─▶ combatLog.push_back(message)
                                     if (size > 10) erase(begin())
```

6.2 渲染流程数据流

```
Game::run() 主循环
    |
    ├─▶ Renderer::renderGame(map, player, enemies)
    |       |
    |       ├─▶ Map::updateExplored(player.x, player.y)  // 更新探索区域
    |       |
    |       ├─▶ 遍历地图网格
    |       |       |
    |       |       ├─▶ Map::isVisible()  // 检查可见性
    |       |       ├─▶ Map::isExplored()  // 检查探索状态
    |       |       ├─▶ Map::getTile()  // 获取地形
    |       |       ├─▶ Player::getX()/getY()  // 玩家位置
    |       |       ├─▶ Enemy::getX()/getY()  // 敌人位置
    |       |       ├─▶ Enemy::getFootprints()  // 敌人足迹
    |       |       └─▶ Player::getFootprints()  // 玩家足迹
    |       |
```

```
    |        └▶ 输出到控制台（带颜色）
    |
    └▶ Renderer::renderPlayerStatus()
            |
            ├▶ Player::getStatus()  // 玩家状态
            ├▶ Enemy::isAlive()  // 敌人存活状态
            ├▶ Enemy::getSymbol()  // 敌人符号
            ├▶ Enemy::getName()  // 敌人名称
            └▶ combatLog  // 战斗日志
```

---

7. 附录

7.1 枚举类型定义

GameResult 枚举

枚举值	值	说明
`VICTORY`	0	玩家胜利
`DEFEAT`	1	玩家失败
`QUIT`	2	玩家退出

Difficulty 枚举

枚举值	值	说明
`EASY`	0	简单模式
`HARD`	1	困难模式

7.2 结构体定义

DifficultySettings 结构体

属性	类型	说明
`enemyCount`	`int`	敌人数量
`enemyHp`	`int`	敌人 HP
`enemyAttack`	`int`	敌人攻击力

难度配置表:

难度	enemyCount	enemyHp	enemyAttack
EASY	4	10	3
HARD	8	15	6

7.3 常量定义

常量名	值	所属类	说明
`Map::WIDTH`	20	Map	地图宽度
`Map::HEIGHT`	20	Map	地图高度
`Map::VIEW_RADIUS`	6	Map	视野半径
`Player::MAX_FOOTPRINTS`	5	Player	最大玩家足迹数
`Enemy::MAX_FOOTPRINTS`	5	Enemy	最大敌人足迹数
`Battle::MAX_COMBAT_LOG_SIZE`	10	Battle	最大战斗日志数
`Game::CHASE_RADIUS`	5	Game	敌人追击半径

7.4 圈复杂度统计表

核心方法	版本	if/else/for/while 总数	McCabe 圈复杂度 (总数 + 1)
登录验证 (showMainMenu)	重构前	10	11
登录验证 (showMainMenu)	重构后	10	11
攻击判定 (combat/executeCombat)	重构前	10	11
攻击判定 (combat/executeCombat)	重构后	10	11

7.5 重构前后对比

指标	重构前	重构后	提升
Game 类代码行数	~500 行	~465 行	↓ 7%
模块数量	1 个上帝类	6 个独立模块	☑ 职责分离
代码复用	玩家/敌人重复代码	Entity 基类共享	☑ 消除重复
可测试性	无法独立测试	各模块可独立测试	☑ 大幅提升
可扩展性	修改一处影响全局	添加新模块不影响旧模块	☑ 开闭原则

文档修订记录

版本	日期	修订人	修订内容
v1.0	2026-03-26	开发团队	初始版本（Sprint 1）
v2.0	2026-04-06	开发团队	重构版本：新增 Entity/Battle/Renderer 类详细设计，补充 5 张 N-S 盒图，规范化方法签名

文档结束

☑ 本文档符合 IEEE 1016-2009 软件详细设计规范，可直接用于实验报告提交和后续开发参考。